Data Analytics Project: IGN Game Reviews Analysis

Dataset Used: ign.csv

This document outlines a structured, end-to-end data analysis project based on game reviews from the IGN dataset. The project is divided into five scenarios, increasing in difficulty from basic data handling to advanced analysis and visualization, ensuring comprehensive skill coverage.

Project Goal: To analyze game review scores, platform popularity, and genre distribution over time using the `ign.csv` dataset.

The **primary libraries/modules** required for this project are:

- **Pandas:** For all data manipulation, filtering, aggregation, and feature engineering tasks.
- **NumPy:** Specifically for numerical calculations like yearly growth (`np.diff`).
- **Visualization Library (e.g., Matplotlib):** For generating all required plots (Line Graphs, Bar Charts, Pie Charts, Stacked Bar Charts).

🎯 Project Coverage

Category	Skills Covered
Data Preprocessing	Data Loading, Missing Value Handling (Mean/Mode), Type Conversion, Column Removal
Pandas Operations	Grouping, Filtering, Aggregation, Feature Engineering (score_category, editors_choice conversion)
NumPy Calculations	Array conversion, Yearly score growth calculation (np.diff)
Visualization	Line Chart, Bar Chart, Pie Chart, Histogram, Stacked Bar Chart, Saving Plots

Scenario 1: Basic Data Loading & Cleaning (□ Easy)

Description:
In this scenario we are loading the dataset ign.csv to the DataFrame. After that we are displaying the first and last 5 rows along with shape of the dataset.and delete the unnecessary column from the dataset.Missing values from score column is replaced with mean value and genre column with the mode. We need to ensure the values have the proper datatypes.

Task	Description
1.	Load the `ign.csv` dataset using Pandas.
2.	Display the first 5 rows (`head()`), last 5 rows (`tail()`), and the shape of the dataset.
3.	Remove the unnecessary column: `"Unnamed: 0"`.
4.	Check for missing values in `score`, `genre`, and `platform`.
5.	Handle missing values: Fill the numeric column `score` with the mean, and the categorical column `genre` with the mode.

Task	Description
1.	Load the `ign.csv` dataset using Pandas.
2.	Display the first 5 rows (`head()`), last 5 rows (`tail()`), and the shape of the dataset.
3.	Remove the unnecessary column: `"Unnamed: 0"`.
6.	Ensure correct data types: Convert `score` → float and `release_year`, `release_month`, `release_day` → integer.

Task Workflow:

The dataset is uploaded to the Dataframe using **read_csv()** fuction once the dataset is loaded we can check the top 5 rows and bottom 5 rows using **head()** and **tail()** functions respectively. To shape of the dataset we canuse **df.shape** attribute.

```
First 5 rows:
   Unnamed: 0 score_phrase  ... release_month release_day
0           0      Amazing  ...             9          12
1           1      Amazing  ...             9          12
2           2        Great  ...             9          12
3           3        Great  ...             9          11
4           4        Great  ...             9          11

[5 rows x 11 columns]

Last 5 rows:
       Unnamed: 0 score_phrase  ... release_month release_day
18620       18620         Good  ...             6          29
18621       18621      Amazing  ...             6          29
18622       18622     Mediocre  ...             6          28
18623       18623  Masterpiece  ...             6          28
18624       18624  Masterpiece  ...             6          28
```

To remove the unnecessary column which don't have any column names we can use **drop()** method.After using drop function, the table won't be having unnamed columns as show.

Conclusion:

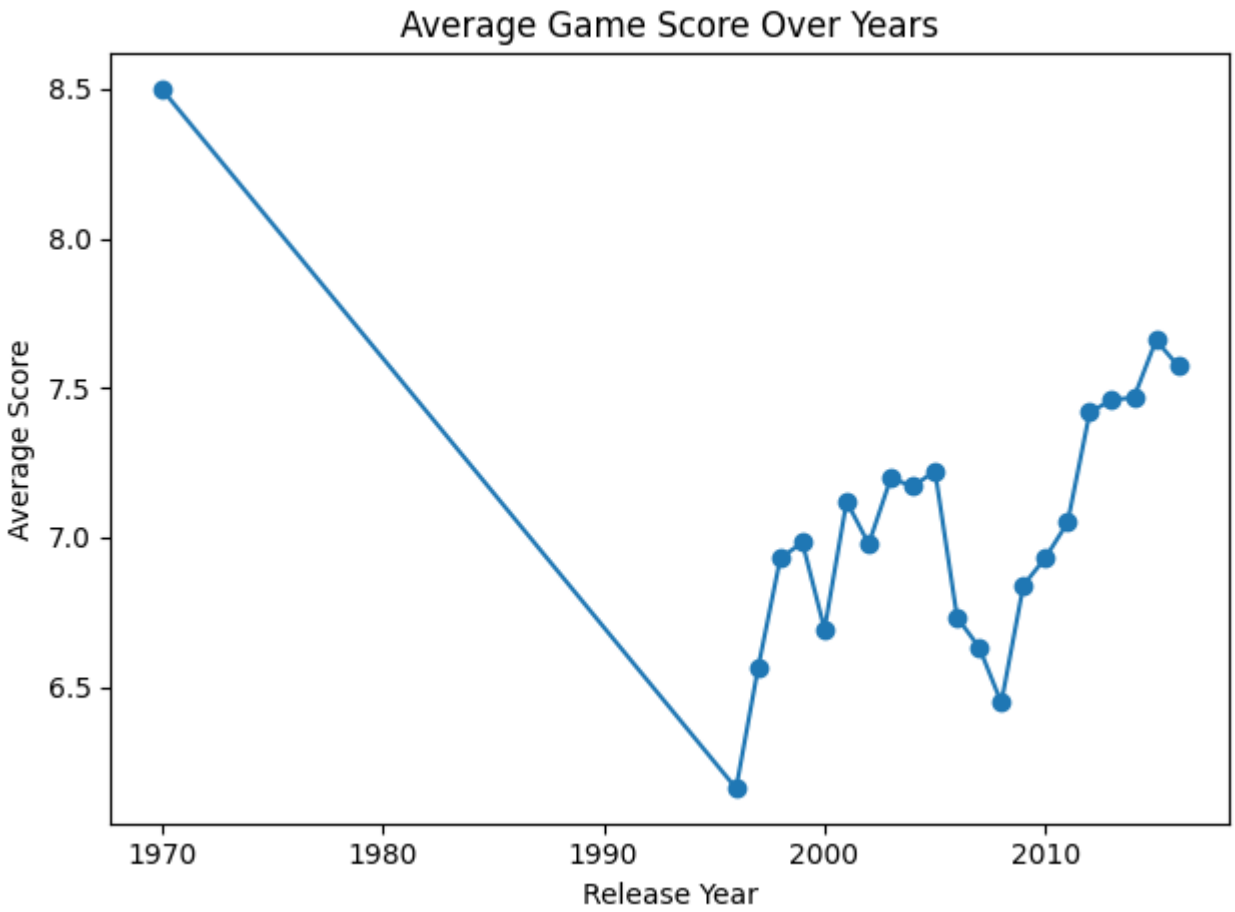
In this scenario we loaded the dataset to the pandas DataFrame. Displayed the top and bottom columns. Cleaned the dataset by removing the unwanted columns and updated the missing cells with proposed values. And also the datatype are upadated as per requriment

Scenario 2: Line Graph (Score Trend) + Save (□ Easy)

Description:
Data loading and preprocessing is the first and most important step in data analysis. In this scenario, the dataset (ign.csv) is loaded into a Pandas DataFrame, which allows the data to be handled in a structured table format. Functions like head(), tail(), and shape are used to understand the dataset by viewing sample rows and checking its size. Unnecessary columns such as "Unnamed: 0" are removed to keep the data clean and relevant. Missing values in important columns like score, genre, and platform are identified and handled properly—numeric values like score are filled with the mean, while categorical values like genre are filled with the mode.

Task	Description
1.	Group data by `release_year`.
2.	Calculate the average score per year using Pandas.
3.	Convert the resulting average scores and years into NumPy arrays.
4.	Plot a line graph with `release_year` on the X-axis and average score on the Y-axis.
5.	Add the title: "Average Game Score Over Years" and proper axis labels.
6.	Save the graph: `plt.savefig("avg_score_trend.png")`.

Graph:



Conclusion:

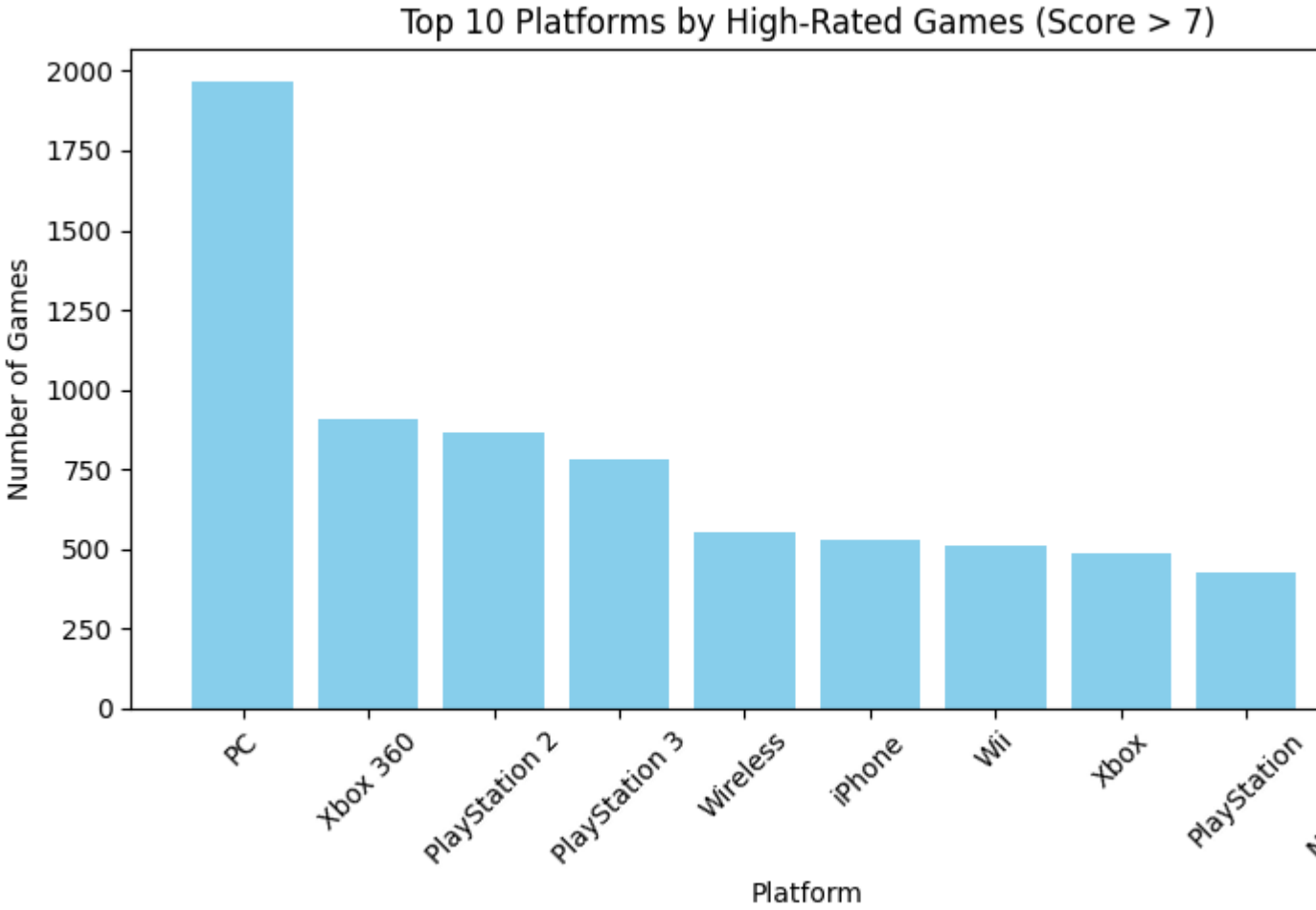
The data grouping by release_year successfully aggregates scores into a digestible annual format. Using Pandas for the initial calculation ensures efficient handling of the mean scores across the dataset.Converting the results to NumPy arrays provides a clean, high-performance format for the Matplotlib library.

Scenario 3: Filtering + Bar Chart + Save (□ Medium)

Description:
This analysis isolates high-quality games by filtering the dataset for scores above 7.0 and identifying which hardware platforms host the most "must-play" titles. By focusing on the top 10 platforms, the resulting bar chart provides a clear visual ranking of industry leaders based on critical acclaim rather than just total volume.

Task	Description
1.	Filter the dataset to include only games where `score > 7`.
2.	Count the number of high-rated games per platform.
3.	Select the top 10 platforms using Pandas.
4.	Convert the platform counts into NumPy arrays.
5.	Plot a bar chart with `platform` on the X-axis and count of games on the Y-axis.
6.	Rotate x-axis labels for readability.
7.	Save the graph: `plt.savefig("top_platforms_bar.png")`.

Graph :



Conclusion:

Filtering for scores > 7 ensures the comparison is based on quality rather than just the quantity of releases.The value_counts() function efficiently ranks the platforms,

highlighting which ecosystems have the strongest library of hits. Utilizing NumPy arrays ensures the plotting process is memory-efficient and compatible with Matplotlib's core functions.

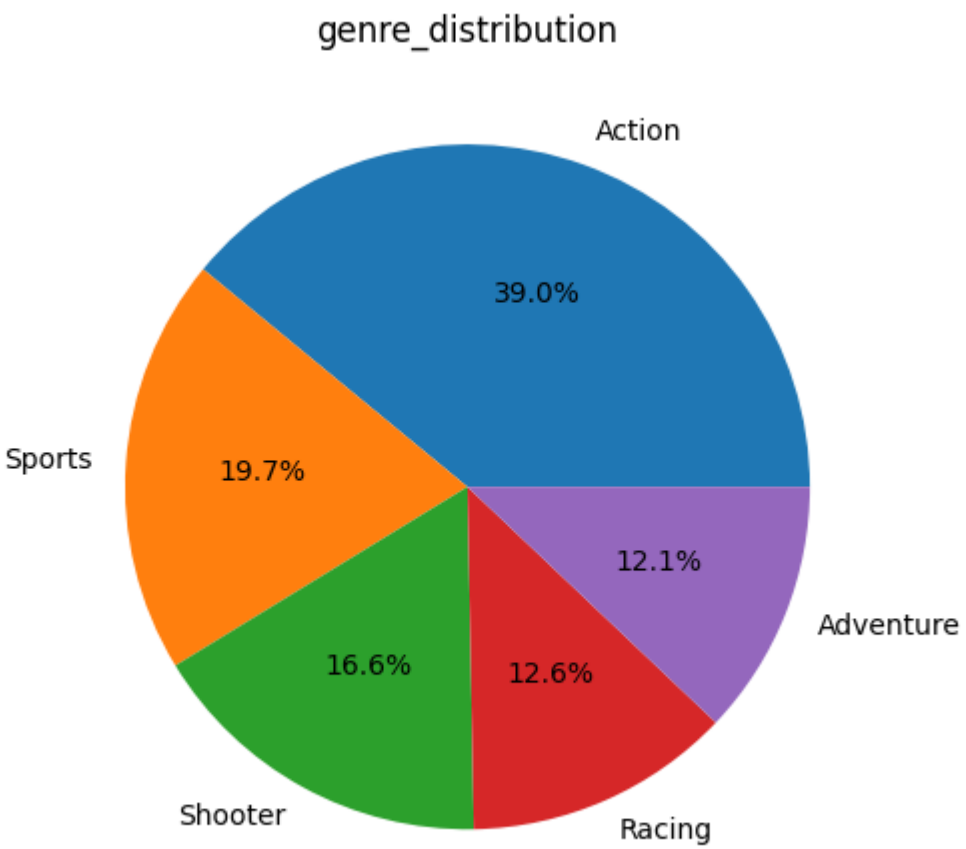
Scenario 4: Aggregation + Pie Chart + Save (□ Medium)

Description:

This analysis utilizes data aggregation to identify the most prevalent game genres within the dataset, specifically isolating the top five categories to maintain visual clarity. By transforming raw categorical data into a proportional format, the pie chart illustrates the market share of each leading genre relative to its peers.

Task	Description
1.	Count the total number of games per `genre`.
2.	Select the top 5 genres using Pandas.
3.	Prepare labels (genres) and values (counts) for the pie chart.
4.	Plot a pie chart using `genre` as labels and `count` as values.
5.	Add percentage labels using `autopct` (e.g., '%.1f%%').
6.	Save the graph: `plt.savefig("genre_distribution.png")`.

Graph :



Conclusion:
Finally, saving the output as a high-resolution image ensures that these insights are preserved for future reporting or collaborative review. This streamlined workflow demonstrates how categorical counting can be effectively translated into a professional-grade distribution summary.

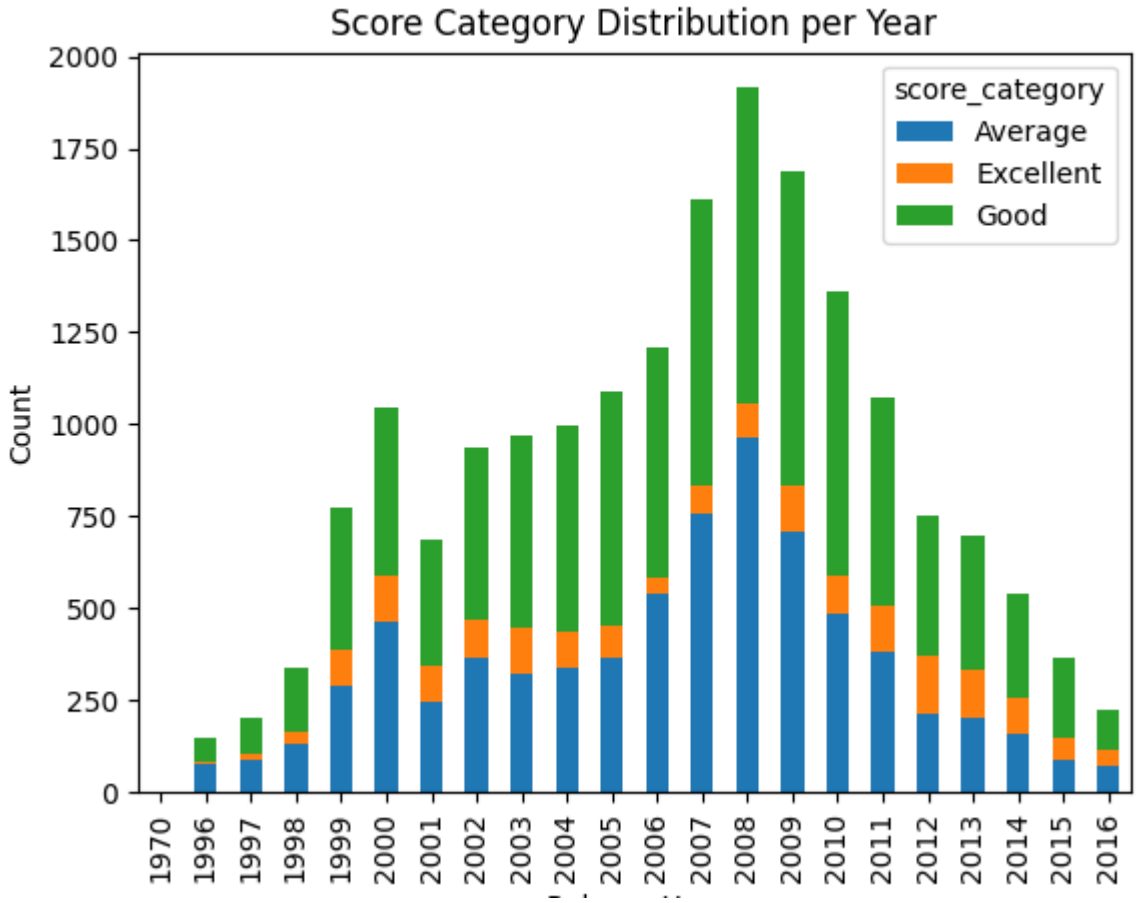
Scenario 5: Advanced Analysis + Multiple Graphs (● Hard)

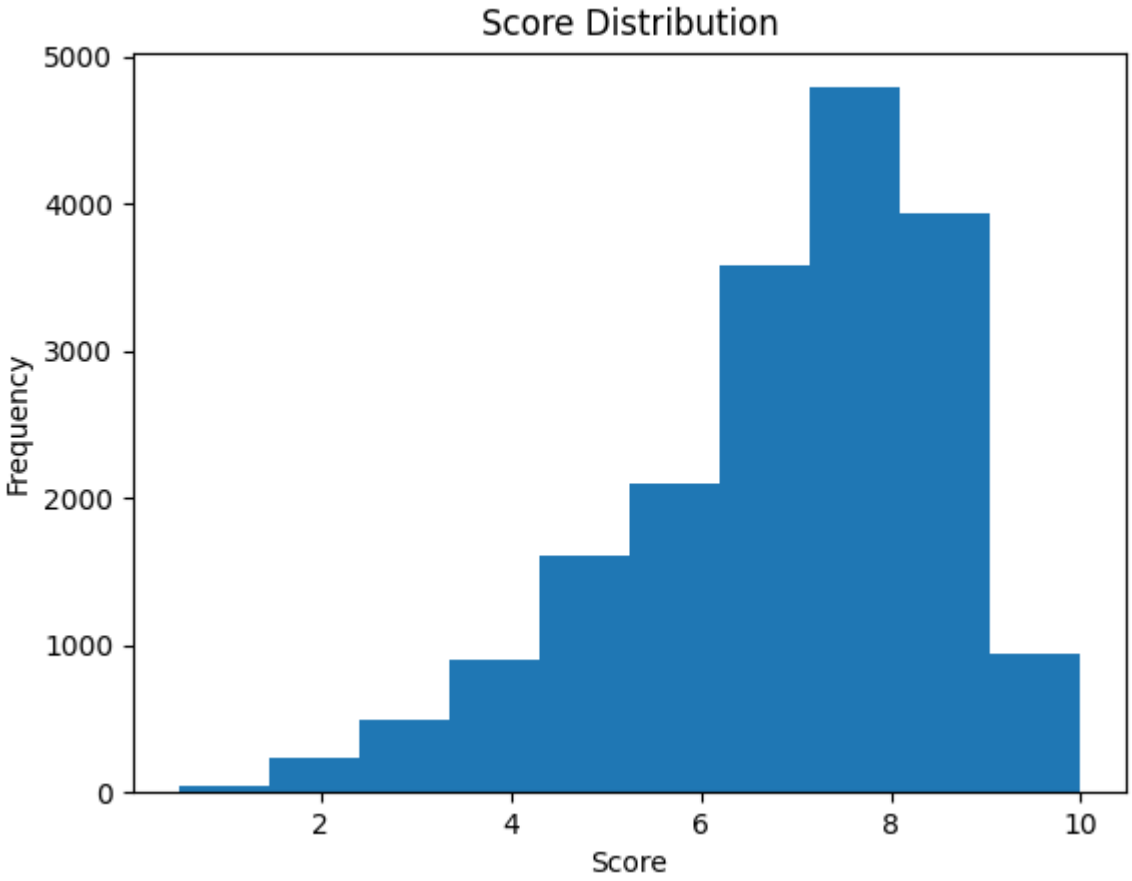
Description:
This analysis focuses on understanding review patterns using feature engineering, NumPy, and multiple visualizations. A new column called score_category is created to classify reviews into Excellent, Good, and Average based on score ranges. The editors_choice column is converted into numerical form (1 and 0) for better analysis. Using NumPy, yearly score growth is calculated from average yearly scores to observe trends.

Task	Description
1.	Create a new column `score_category`: "Excellent" for score ≥ 9, "Good" for $7 \le$ score < 9, and "Average" for score < 7.
2.	Convert the `editors_choice` column: 'Y' $\rightarrow 1$, 'N' $\rightarrow 0$.
3.	Use NumPy's `np.diff()` to calculate the **yearly score growth** based on the average yearly scores from Scenario 2.
4.	Plot a **Line Graph** showing the trend of average score per `release_year`.
5.	Plot a **Stacked Bar Chart** showing the count of `score_category` per `release_year`.
6.	Plot a **Histogram** showing the distribution of `score`.
7.	Save the graphs: `plt.savefig("score_trend.png")`, `plt.savefig("score_category_stacked.png")`, `plt.savefig("score_distribution.png")`.
8.	Identify: Which years had the highest scores, whether high scores increased over time, and if `editors_choice` correlates with high scores.

Graph :

Average Game Score Over Years
8.5
8.0
7.5
7.0
6.5
Average Score
1970
1980
1990
2000
2010
Release Year


Score Category Distribution per Year
score_category
Average
Excellent
Good
Count
0
250
500
750
1000
1250
1500
1750
2000
1970
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016



Conclusion:

The analysis shows that certain years have higher average scores, indicating strong game releases during those periods. The line graph suggests that scores have remained relatively stable with slight fluctuations rather than a continuous increase. The stacked bar chart reveals that “Good” and “Excellent” categories dominate in most years. The histogram indicates that most scores fall between 7 and 9, showing generally positive reviews. Editors’ choice selections are mostly associated with higher scores, suggesting a strong correlation

Final Conclusion:

The analysis of game reviews shows that average scores vary across years, with certain years achieving higher ratings as seen in the trend graph. Although there are some fluctuations, the overall pattern

suggests a slight improvement or stability in game quality over time. The distribution of score categories indicates that most games fall under the “Good” range, while “Excellent” games appear less frequently but consistently each year. The histogram further confirms that scores are mainly concentrated between 7 and 9. Additionally, games marked as Editors’ Choice tend to have higher average scores compared to others, showing a strong positive correlation between editorial selection and game quality

What you have learned

Through this analysis, I learned how to perform complete data processing and visualization using Python. I understood how to create new features like score categories based on conditions and convert categorical data into numerical form for analysis. I also learned to use NumPy functions such as np.diff() to measure changes in average scores over time. By creating different visualizations like line graphs, stacked bar charts, and histograms, I gained insight into trends, distributions, and comparisons within the data. Additionally, I learned how to save graphs for reporting purposes. Most importantly, I developed the ability to interpret results, such as identifying trends in game scores over the years, understanding the distribution of ratings, and recognizing the strong relationship between Editors’ Choice and higher scores. Overall, this task helped me build practical skills in data analysis, visualization, and drawing meaningful conclusions from data.